



Introduction



Introduction

Foundations

- Physical AI Trends
- Introduction to OpenUSD

Isaac Sim 101

- Isaac Sim core essentials
- Basic Tutorial

CAD to Sim

- Robot Rigging
- Task Control

World Model

- NVIDIA Cosmos
- Synthetic Data Generation

Hands on

Hands on: Train an SO-101 Robot From Sim-to-Real With NVIDIA Isaac

Physical AI Trends

Figure 1 - Helix



본 문서에서 인용된 모든 자료는 원 저작자에게 허락을 받아 제작되었으며, 본 문서의 수정, 편집 배포는 제한됩니다.

Overview

Graphics

USD, CAD, URDF 등에 대한 이해
Digital Twin의 시각적/ 구조적 완성

Physics

NVIDIA Warp, CUDA 기반 물리 해석 및 CAE, 제어공학적 지식 활용

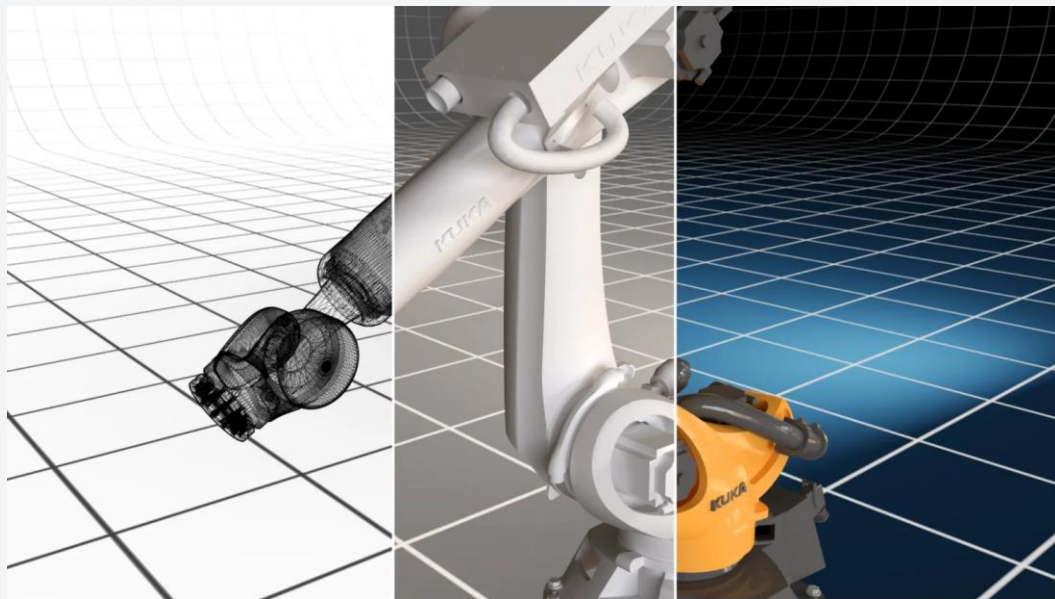
Software

Python을 기반으로 하는 시뮬레이션 라이프사이클 관리, AI 모델 사용 능력

Graphics

OpenUSD(Universal Scene Description): 그래픽 정의 및 파이프라인을 구성하는 프레임워크

OMNIVERSE CAD Converter: 산업용 디자인을 SimReady Asset으로 변경



Physics

NVIDIA Warp: 미분 가능한 물리(Differentiable Physics) 커널 작성을 위한 프레임워크

CUDA Acceleration: GPU 커널 최적화로 하드웨어 활용 최적화

Mechanics Modeling: 시뮬레이션 내부 로봇 구현에서의 정역학 및 동역학

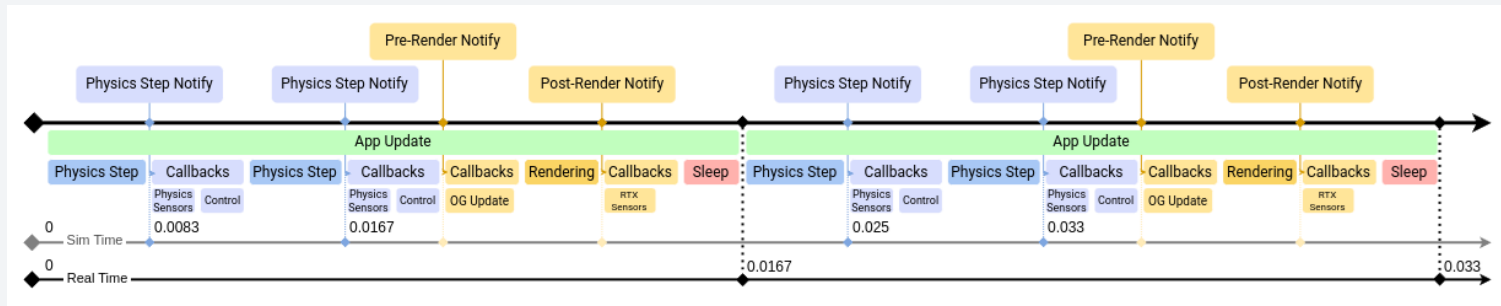


Software

Basic Coding Experience: 딥러닝 프레임워크를 포함하는 기본적인 코딩 능력

Simulation Step & Loop: 물리엔진 Step에 대한 이해

Lifecycle: 위 내용에 더해 시뮬레이션 내부 객체의 생성, 소멸 등에 대한 이해



Traditional Robotics

Rule Based execution:

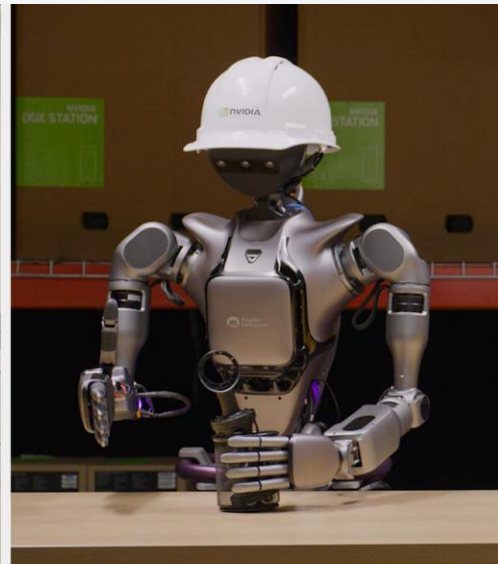
- Hard-coded trajectories with zero adaptability
- Designed for repeatability
- > Highly vulnerable to noises



Robots with Physical AI

Policy Based execution:

- Generative trajectories with full adaptability
- Designed for generalization
- > Robust noise grounding



NVIDIA OMNIVERSE

NVIDIA Omniverse™는 산업용 디지털 트윈과 로봇틱스 시뮬레이션 같은 **피지컬 AI** 애플리케이션을 개발하기 위한 라이브러리와 마이크로서비스 모음입니다.

- 기존 5개 기둥(Nucleus, Connect, Kit, Simulation, RTX Renderer)을 기반으로 하는 프레임워크
- 현재는 규모가 커지고 활용 분야가 다양해짐에 따라 정의가 변화하였음



NVIDIA OMNIVERSE

[NVIDIA Omniverse Libraries \(Link\)](#)

Home / NVIDIA Omniverse

NVIDIA Omniverse Libraries

Omniverse is NVIDIA's collection of libraries and microservices for physical AI development.

Scroll to Resource

[Learning Library](#)

Quick Links:

- > Documentation
- > Developer Guide
- > Learning Path
- > Join Community
- > Forum
- > Discord

NVIDIA OMNIVERSE

Omniverse Core Libraries and SDKs

ovrtx (Early Access)

GPU-accelerated, physically based rendering and sensor-simulation libraries built on NVIDIA RTX™.

Get Started >

ovphysx (Early Access)

USD-native, open source multiphysics library for scalable robotics and digital twin simulation.

Get Started >

ovstorage (Early Access)

Cloud-native APIs for managing, storing, and synchronizing OpenUSD asset data.

Get Started >

Kit Framework SDK

Core libraries, templates, and developer tooling for building custom physical AI applications.

Explore Docs >

NVIDIA OMNIVERSE [Blueprints](#)

Get Started With Developer Blueprints



“DSX” Gigawatt-Scale AI Factories

Optimize design, simulation, and operations of AI factories with NVIDIA libraries.

[Try Blueprint >](#)

[Watch Video >](#)



“GROOT-Mimic” Synthetic Robot Manipulation

Generate exponential synthetic manipulation data for robots.

[Try Blueprint >](#)

[Watch Video >](#)



“MEGA” Multi-Robot Industrial Fleets Automation

Simulate complex industrial robot fleets before real-world deployment.

[Try Blueprint >](#)

[Watch Video >](#)



Physical AI Data Factory

Accelerate robotics, vision AI, and autonomous vehicle development with physical AI data.

[Read the Press Release >](#)

NVIDIA OMNIVERSE

Omniverse Learning Resources

Showing 57 results

⏪ ⏩ 1 ▾ ⏪ ⏩

Featured

Tutorial

Build and Orchestrate End-to-End SDG Workflows...

This post explores creating a simulated environment, generating synthetic data with MobilityGen on...

Featured

Synthetic Manipulation Motion Generation for...

Synthetic data from accurate simulations speeds up data collection and adds diversity, solvin...

Featured

News

Cosmos | Isaac Lab | Isaac Sim

Developers Build Fast and Reliable Robot Simulation...

NVIDIA Omniverse libraries and Cosmos WFMs, powered by OpenUSD, give developers tools to...

OMNIVERSE KIT SDK

Overview — Omniverse Kit

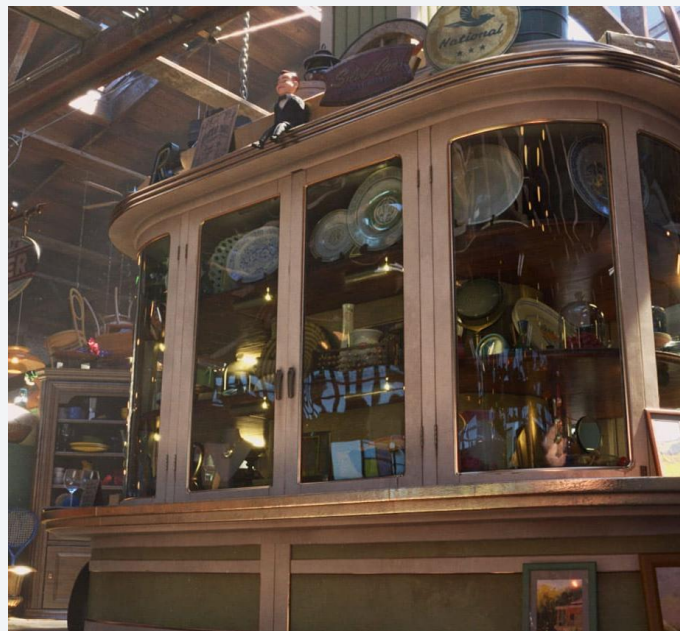
다양한 Omniverse Application 개발을
지원하는 SDK

- Isaac Sim
- USD Explorer
- 이외에도 모듈형 익스텐션 구조로 개발
가능

OpenUSD

OpenUSD(Universal Scene Description)는 3D 데이터를 설명하는 프레임워크

- Pixar가 2016년에 OpenUSD를 공개하였고, AOUSD가 2023년도부터 출범하여 관리 중
- 복잡한 3D Data 협업의 표준



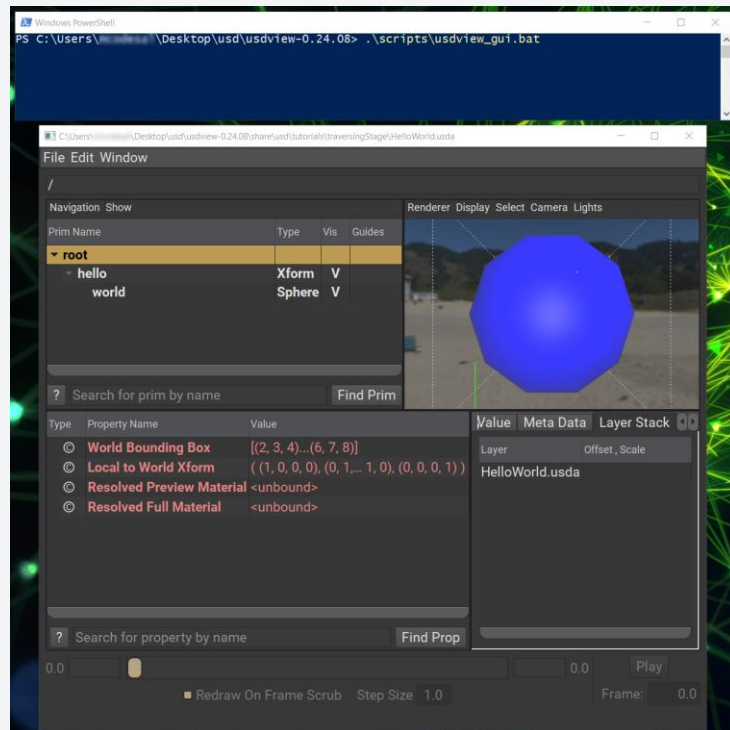
Toy Story 4 (2019)

UsdView

USD 파일을 가볍고 빠르게 열어볼 수 있는 뷰어

[usdview Quickstart — Omniverse USD](#)

위 페이지를 참고하여 설치해보기

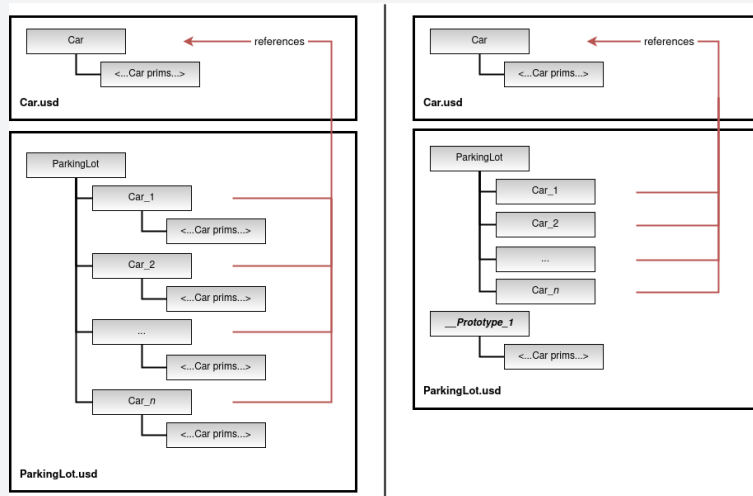


Stage

<https://docs.omniverse.nvidia.com/dang/latest/guide/usd/usd-fundamentals.html>

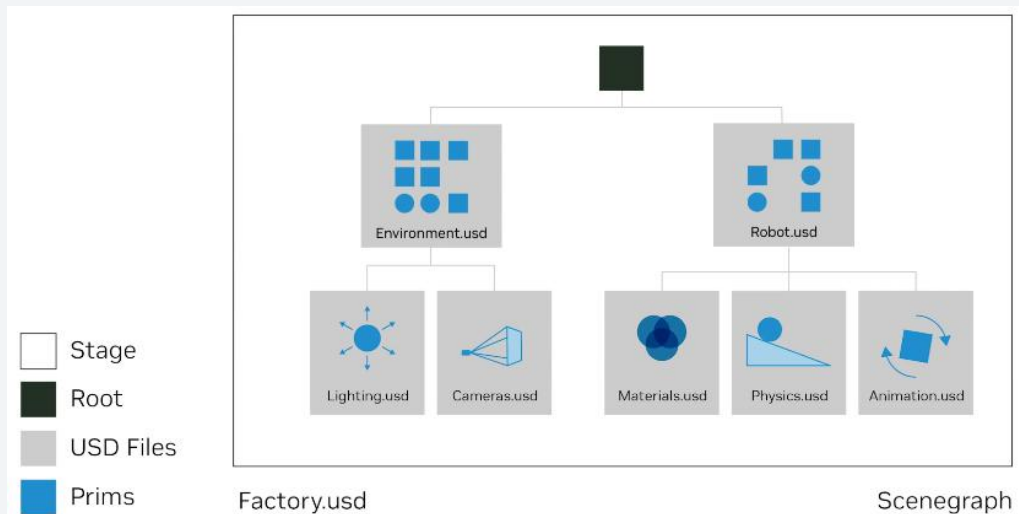
Stage는 layer들을 합성한 결과물로, 현재 화면 출력 결과물

- Layer: 보통 하나의 .usd파일 단위
- Composition: 여러 레이어의 결합



SCENEGRAPH STRUCTURE

Stage는 scene에 무엇이 있는지 나타내는 계층구조인 scenegraph를 제공합니다



실습: Stage

```
# usdview basic settings
```

```
stage = usdviewApi.stage
```

```
stage.SetEditTarget(stage.GetRootLayer())
```

```
stage.RemovePrim('/hello')
```

```
stage.ClearDefaultPrim()
```

```
stage.Save()
```

실습: Stage

```
stage.DefinePrim("/World", "Xform")
```

```
# print the stage as string
```

```
print(stage.ExportToString(addSourceFileComment=False))
```

Prims

Prim은 Primitives의 약자로, scene 내부 객체의 data, attributes, relationships를 포함하는 컨테이너

- Unique Path: Prim을 구별하는 요소(예: /World/Environment)
- Imageable/Non-Imageable

Prim

```
# Import the Sdf class
```

```
from pxr import Sdf
```

```
# Return the path of a Usd.Prim as an Sdf.Path object
```

```
Usd.Prim.GetPath()
```

```
# Retrieve a Usd.Prim at the specified path from the Stage
```

```
Usd.Stage.GetPrimAtPath()
```

실습: Prim

```
# Import the `Usd` module from the `pxr` package:
```

```
from pxr import Usd
```

```
# Define a new primitive at the path "/hello" on the current stage:
```

```
stage.DefinePrim("/hello")
```

```
# Define a new primitive at the path "/world" on the current stage with the  
prim type, Sphere.
```

```
stage.DefinePrim("/world", "Sphere")
```


실습: Stage

```
# stage.DefinePrim(prim_path, GPrim)
```

GPrims

Cube (정육면체)

Sphere (구)

Cylinder (원기둥)

Capsule (알약 모양)

Cone (원뿔)

실습: Prim

```
from pxr import UsdGeom

# Define a prim of type `Sphere` at path `/hello`:
sphere: UsdGeom.Sphere = UsdGeom.Sphere.Define(stage, "/hello")

sphere.CreateRadiusAttr().Set(2)

# Remove a prim from the stage

stage.RemovePrim("/hello")
```

실습: Prim

```
# Define a Scope prim in stage at `/Geometry`
```

```
geom_scope: UsdGeom.Scope = UsdGeom.Scope.Define(stage, "/Geometry")
```

```
# Define an Xform prim in the stage as a child of /Geometry called GroupTransform
```

```
xform: UsdGeom.Xform = UsdGeom.Xform.Define(stage,  
geom_scope.GetPath().AppendPath("GroupTransform"))
```

```
# Define a Cube in the stage as a child of /Geometry/GroupTransform, called Box
```

```
cube: UsdGeom.Cube = UsdGeom.Cube.Define(stage, xform.GetPath().AppendPath("Box"))
```

실습: Prim

```
prim: Usd.Prim = stage.GetPrimAtPath("/Geometry")
child_prim: Usd.Prim
if child_prim := prim.GetChild("Box"):
    print("Child prim exists")
else:
    print("Child prim DOES NOT exist")
```

Attributes

이름-값 (name-value) 형태로 특정 값(data type)을 가지는 Property입니다.

간단한 개념인 만큼 Python code도 제일 많이 사용

```
# Prim의 Property name 받아오기
```

```
prop_names = prim.GetPropertyNames()
```

```
# UsdGeom.Sphere 객체의 attribute get, set
```

```
sphere_prim.GetRadiusAttr().Get()
```

```
sphere_prim.GetDoubleSidedAttr().Set(True)
```

실습: Attributes

```
# Retrieving Properties of a Prim
```

```
from pxr import Usd, UsdGeom, Gf
```

```
world_xform: UsdGeom.Xform = UsdGeom.Xform.Define(stage, "/World")
```

```
# Define a sphere under the World xForm:
```

```
sphere = UsdGeom.Sphere.Define(stage, world_xform.GetPath().AppendPath("Sphere"))
```

```
# Define a cube under the World xForm and set it to be 5 units away from the sphere:
```

```
cube: UsdGeom.Cube = UsdGeom.Cube.Define(stage, world_xform.GetPath().AppendPath("Cube"))
```

```
UsdGeom.XformCommonAPI(cube).SetTranslate(Gf.Vec3d(5, 0, 0))
```

실습: Attributes

```
# Get the property names of the cube prim:
```

```
cube_prop_names = cube.GetPrim().GetPropertyNames()
```

```
# Print the property names:
```

```
for prop_name in cube_prop_names:
```

```
    print(prop_name)
```

실습: Attributes

```
# Get the attributes of the cube prim
cube_attrs = cube.GetPrim().GetAttributes()
for attr in cube_attrs:
    print(attr)

# Get the size, display color, and extent attributes of the cube
cube_size: Usd.Attribute = cube.GetSizeAttr()
cube_displaycolor: Usd.Attribute = cube.GetDisplayColorAttr()
cube_extent: Usd.Attribute = cube.GetExtentAttr()

print(f"Size: {cube_size.Get()}")
print(f"Display Color: {cube_displaycolor.Get()}")
print(f"Extent: {cube_extent.Get()}")
```


실습: Attributes

```
# Get the size, display color, and extent attributes of the cube
```

```
cube_size: Usd.Attribute = cube.GetSizeAttr()
```

```
cube_displaycolor: Usd.Attribute = cube.GetDisplayColorAttr()
```

```
cube_extent: Usd.Attribute = cube.GetExtentAttr()
```

```
# Modify the size, extent, and display color attributes:
```

```
cube_size.Set(cube_size.Get() * 2)
```

```
cube_extent.Set(cube_extent.Get() * 2)
```

```
cube_displaycolor.Set([(0.0, 1.0, 0.0)])
```

Relationship

Usd Properties 중 SdfPath를 가지는 링크속성

- 객체간의 관계 정의
- Reference 등..

Isaac Sim에서 Reference를 추가하는 방법

<https://docs.isaacsim.omniverse.nvidia.com/5.1.0/py/source/extensions/isaacsim.core.utils/docs/index.html#module-isaacsim.core.utils.stage>

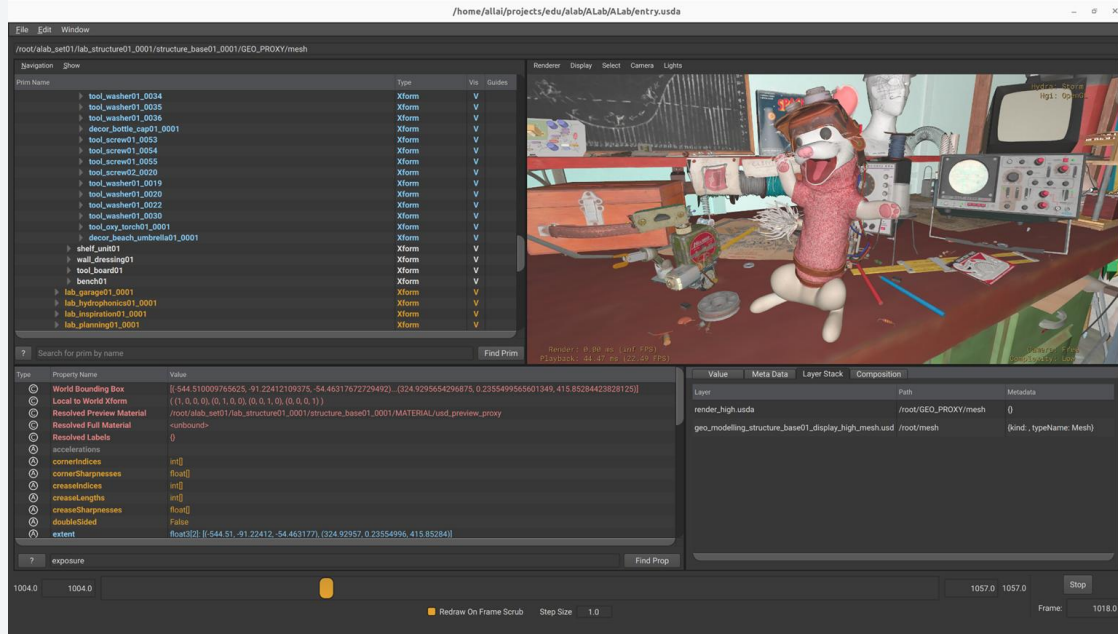
TimeCodes and TimeSamples

애니메이션을 위한 시간(TimeCode) 및 값(TimeSample) 저장 방식입니다.

- Isaac Sim이 PhysX 엔진을 사용하는 물리 시뮬레이션 툴이지만, 특정 애니메이션이 필요할 수 있음
- 코드를 사용하여 작업할 경우 linear interpolation 자동 적용
- 비선형 결과물은 DCC 툴로 작업

실습: TimeCodes and TimeSamples

usdview에서 alab을 연 후
play버튼을 눌러 재생해보기



실습: TimeCodes and TimeSamples

```
cube_xform_api = UsdGeom.XformCommonAPI(cube)

cube_xform_api.SetScale(Gf.Vec3f(5, 5, 0.1))

cube_xform_api.SetTranslate(Gf.Vec3d(0, 0, -2))

stage.SetStartTimeCode(1)

stage.SetEndTimeCode(60)
```

실습: TimeCodes and TimeSamples

```
sphere: UsdGeom.Sphere = UsdGeom.Sphere.Get(stage, "/World/Sphere")
sphere_xform_api = UsdGeom.XformCommonAPI(sphere)

# Set translation of the sphere at time 1
sphere_xform_api.SetTranslate(Gf.Vec3d(0, 5.50, 0), time=1)

# Set translation of the sphere at time 30
sphere_xform_api.SetTranslate(Gf.Vec3d(0, -4.50, 0), time=30)

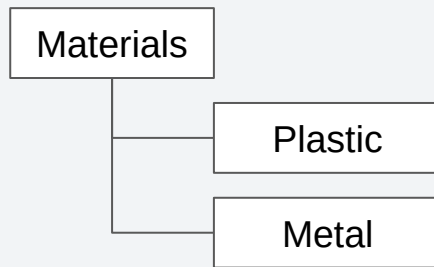
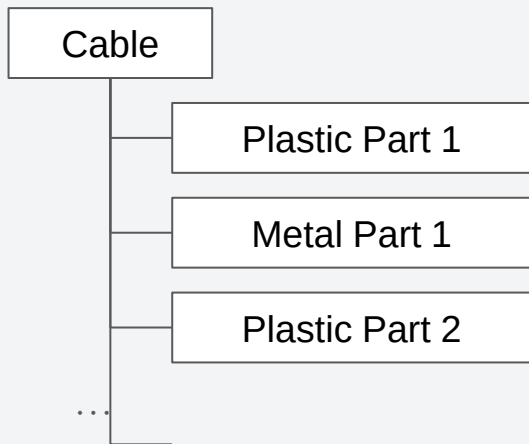
# Set translation of the sphere at time 45
sphere_xform_api.SetTranslate(Gf.Vec3d(0, -5.00, 0), time=45)

# Set translation of the sphere at time 50
sphere_xform_api.SetTranslate(Gf.Vec3d(0, -3.25, 0), time=50)

# Set translation of the sphere at time 60
sphere_xform_api.SetTranslate(Gf.Vec3d(0, 5.50, 0), time=60)
```

Scope

Material 등을 각 Prim에 반복 적용 하기 쉽도록 Scope에 정렬할 수 있습니다.



Xform

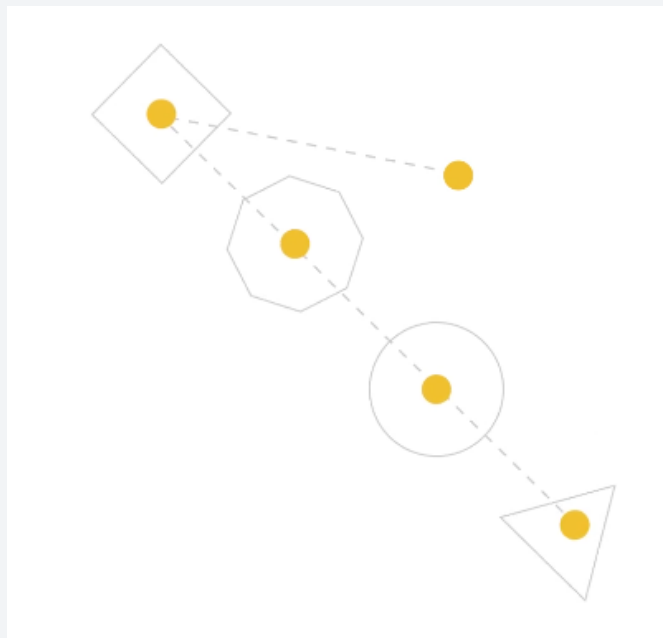
Xform은 Spatial transformations를 뜻합니다.

- Translation
- Rotation
- Scale

Parent의 Transform이 Child에게 영향을 줌

Rotation은 euler, quaternion이 있으며, 상황에 따라

변환 필요



Metadata

Properties에서 Attribute와 Relationship 등을 제외한 나머지 전역 정보

metadata는 시간에 따라 불변하는 값을 주로 사용하나, 필요하다면 사용 가능

대표적인 metadata로 setactive, interpolation이 있음

Lighting

Usd에서 빛은 UsdLux 도메인으로 관리되며 몇 가지 빛의 종류를 정의한다

- DistantLight
- AreaLights
 - CylinderLight
 - RectLight
 - DiskLight
 - SphereLight
- DomeLight
- PortalLight

Modules

OpenUSD 개발을 위한 모듈

OpenUSD가 간단한 확장자가 아닌 프레임워크라고 설명하는 뒷받침

대표적인 모듈로는 Usd, Sdf, Gf가 있다.

isaac sim에서 제공하는 USD 모듈의 목록

[Modules — Omniverse Kit](#)

Primvars

Primvar를 사용해서 복잡한 3D scene에서의 hierarchical object data manipulation 및 management를 할 수 있다.

Isaac Sim을 사용 할 때 Primvar는 보통 Display color, interpolation 등 rendering 시 관련된 속성

File Formats

.usdc (compressed binary format): Lightweight, faster loading

.usda (ASCII text file): Human readable, easily overwrite

.usd: (either usda or usdc, mostly usdc)

.usdz (zipped archive): zipped format to deliver project data - 특별히 온라인 asset을 reference로 포함하는 경우가 있을 수 있으니, 로컬 파일로 치환해야 함

Composition

Stage를 표현할 때 여러 계층의 "opinion"을 합쳐서 렌더링 결과물을 산출해야 함
합치는 기법을 Composition Arc라고 하며, 강한 Opinion을 내는 순서대로 결정됨

LIVRPS - [What Is LIVERPS? — Learn OpenUSD](#)

Isaac Sim에서는 LI를 거의 사용하지 않으며, 대부분 RP로 구성 요소를 불러오게 됨.

INTEROPERABILITY

OpenUSD는 표준화된 C++, Python 라이브러리를 통해 정보를 표현할 수 있고, 다양한 DCC(Digital Content Creation) 툴과 호환 가능

하나의 3D Scene을 여러 파일 구성으로 나누고 결합할 수 있어 협업에 용이

NON-DESTRUCTIVE EDITING

원본 데이터를 훼손하지 않는 Composition 방식을 사용

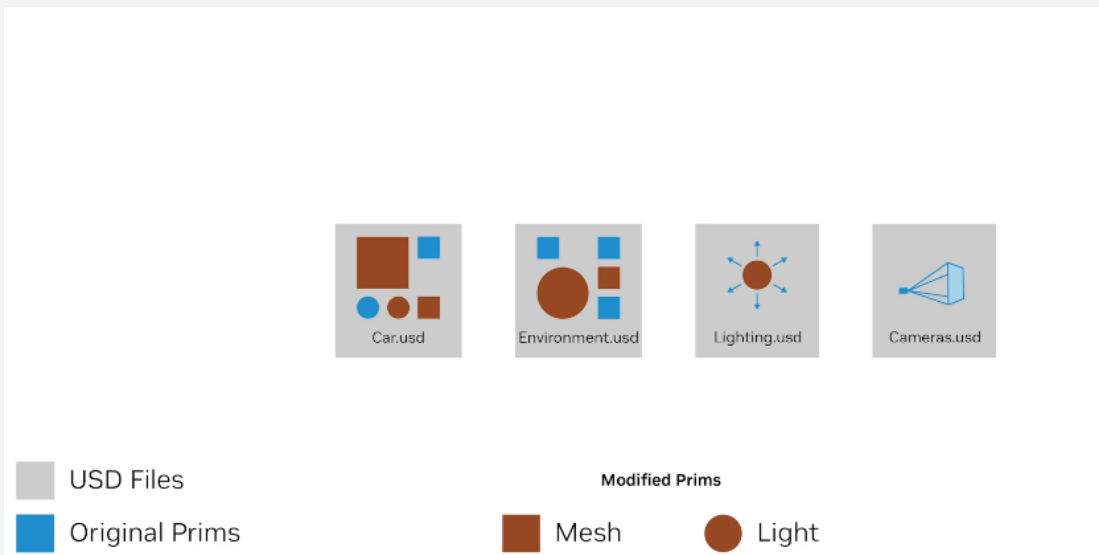
- Layer 기반 수정사항 오버라이드
- 협업 시 작업 영역을 분리

EXTENSIBILITY

표준을 유지한 채로 Custom Schema나 Plugin 등을 연결하여 확장 가능
(RTX renderer, usd physics - Omniverse PhysX 연동)

MODULARITY & SCABILITY

.usd 파일 별 개발이 가능(non-destructive)하고, stage 내부의 .usd파일을 관리할 수 있으므로 stage 규모가 커져도 쉽게 관리가 가능합니다.



INTERACTIVE vs STATIC STATE

USD는 기본적으로 특정 시점의 정적인 “상태”를 저장합니다.

- ‘interactive’한 런타임 값을 저장하지 않음
- 정교한 Scene 구현 시 메모리 효율성 발휘
- Isaac Sim과 같은 앱이 런타임 인터랙션을 담당

Capturing Objects

KIRI Engine이나 Reality Composer와 같은 어플리케이션을 사용해서 핸드폰으로 간단한 오브젝트 캡처가 가능하다

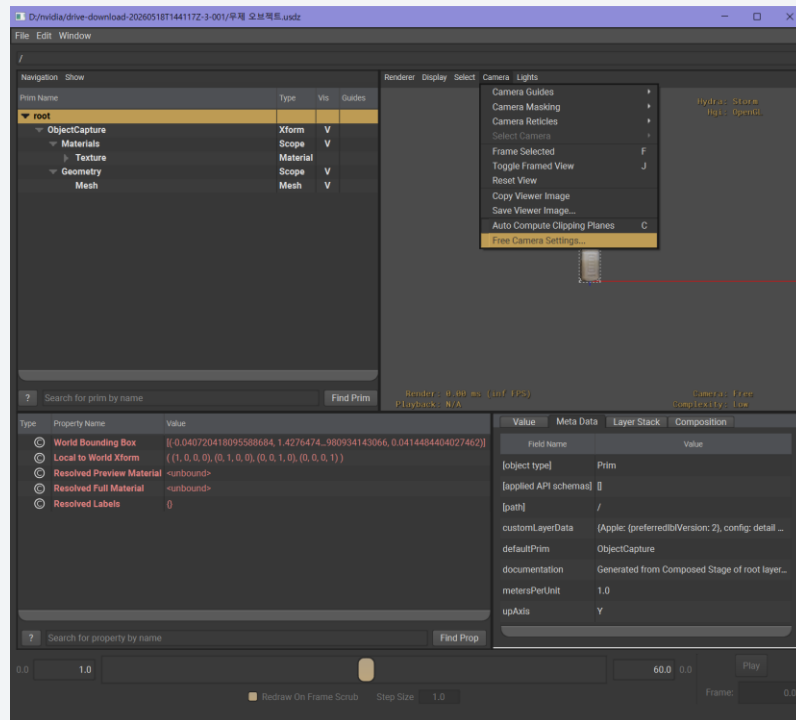
오브젝트를 직접 캡처해보고, 불러보기 (무제 오브젝트.usdz)

Capturing Objects

캡처 파일이 잘 보이지 않을 경우 우측 사진을 참고하여 FOV 조정

Camera > Free Camera Settings...

> Fov 값 낮추기



추가자료

[Kitchen Set — Universal Scene Description 26.05 documentation](#)

[City Set — Universal Scene Description 26.05 documentation](#)

[UsdSkel Examples — Universal Scene Description 26.05 documentation](#)